P2451R0
2021 September Library Evolution Poll Outcomes

Published Proposal, 2021-09-28

Author:
Bryce Adelstein Lelbach (he/him/his) — Library Evolution Chair (NVIDIA)

Source:
GitHub

Issue Tracking:
GitHub

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:
WG21

Table of Contents

Informative References

1. Introduction

In September 2021, the C++ Library Evolution group conducted a series of electronic decision polls [P2436R0]. This paper provides the results of those polls and summarizes the results.

In total, 23 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send [P2418R0] (Add Support For `std::generator`-like Types To `std::format`) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.	12	7	0	1	0	Consenus in favor.
Poll 2: Send [P2415R1] (What Is A `view`?) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.	7	6	1	1	3	Consensus in favor.
Poll 3: Send [P2432R0] (Fix `istream_view`) to Library Working Group for C++23, classified as an improvement of an existing feature	6	8	4	3	1	Weak consensus in favor.

([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.						
Poll 4: Send [P2351R0] (Mark All Library Static Cast Wrappers As `[[nodiscard]]`) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	7	9	4	2	2	Weak consensus in favor.
Poll 5: Send [P2291R2] (Add `constexpr` Modifiers To Functions `to_chars` And `from_chars` For Integral Types In `<charconv>` Header) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	13	7	1	0	0	Strong consensus in favor.

3. Selected Poll Comments

3.1. Poll 1: [P2418R0] Add Support For `std::generator`-like Types To `std::format`

> This fixes cooperation between major C++20 features (ranges and format). Without out, some ranges will be inherently not printable.
>
> — Strongly Favor

> Without this change it will be impossible to format `std::generator` and other types that are neither const-iterable nor copyable.
>
> — Strongly Favor

> An improvement, but yet another one that is not really suitable for a DR at this stage.
>
> — Weakly Favor

> I'll all for fixes to C++20 (and I think this is one). In my mind, C++20 isn't set in stone until C++23 is close to release.
>
> Also, I am quite fine to throw bitfields under the bus. Always.
>
> — Weakly Favor

> I support the general direction of the paper, but I think it's too late for us to continue retroactively changing C++20. I would prefer we instead make a breaking change in C++23. We need to have clear deadlines and releases, but we shouldn't be afraid to fix our mistakes. The more changes we make to C++20, the slower C++20 adoption will be.
>
> — Weakly Against

3.2. Poll 2: [P2415R1] What Is A `view`?

> The view concept has seen so many changes since its initial proposal that its meaning has become muddled. This paper brings some much-needed clarity and is also a significant usability improvement.
>
> — Strongly Favor

> I see the essential feature of a view as "a thing that refers to another thing," rather than its asymptotic complexity. The `shared_ptr` example of LWG3452 is compelling. Making a thing A refer to another thing B would best happen at A's construction, so it makes sense to me to change the wording to focus on construction, rather than destruction. `owning_view` sounds a bit funny ("owning" and "view" feel like opposites), but it does neatly solve the problem of dangling rvalue non-view ranges.
>
> — Strongly Favor

> The O(1) requirement in views is problematic, this is a step in the right direction
>
> — Weakly Favor

> The ranges library is in general overconstrained, requiring that user-supplied types be "well-behaved" in a complete, abstract sense rather than that they actually behave in use. Performance requirements in particular are questionable, since the library cannot act in reliance on them other than via a multiplicative factor.
>
> — Weakly Favor

> I am definitely in favor of the change to the complexity requirements. I don’t fully understand the other changes or the ramifications of them.
>
> — Neutral

> This change would do serious and lasting damage to the design of `std::ranges`, further muddying the distinction between view and ranges, and opening subtle new opportunities for dangling references. No, no, no.
>
> — Strongly Against

> I really believe that rvalue/lvalue difference for meaning of `views:all` is too subtle, and we will pay for it in year. And I found it unfortunate that this change is bundled, with very useful and welcomed clarification on meaning of view.
>
> — Strongly Against

> There are subtleties here that I don’t understand and make me uncomfortable participating.
>
> — Did Not Participate

> I want to participate in this poll, but I don’t think I have the time/knowledge/experience with ranges to do so.
>
> I think it probably makes sense, but I’m not sure of all the ramifications. I would probably go with whatever Eric and Casey say. If the people with experience (ie the paper authors and the ranges authors) all agree with it than great (but I haven’t followed enough to even know their opinions).
>
> — Did Not Participate

§ 3.3. Poll 3: [P2432R0] Fix `istream_view`

> This is an annoying and preventable inconsistency, and we should use the chance to fix this while we still can.
>
> — Strongly Favor

> It's unfortunate to do that post-fact, and it should have been fixed years ago, but better have some (controlled) pain now as opposed to ongoing pain over the next decade or so.
>
> — Strongly Favor

> I really strongly encourage consistency. This problem will not impact large amount of users, and by the same reasoning, change that late is not breaking a lot of code.
>
> — Weakly Favor

> I'm all for fixing C++20 now, before it gets used by lots an lots of people. Adoption is always slow, we can continue to fix things.
>
> — Weakly Favor

> SF for adding `views::istream`. WA on changing `istream_view` (once we have `views::istream`, nobody should be using `istream_view`; it seems unnecessarily breaking all things considered). WF overall.
>
> — Weakly Favor

> Consistency by itself is a poor reason to change something in C++20, but given that there are numerous other retroactive changes being made, this change is a reasonable candidate for that set.
>
> — Neutral

> Consistency in the standard library would be nice, but [my users] don't care about binary compatibility and will use whatever API is available at the time we need the feature.
>
> — Neutral

> These are not things that need to be applied as a DR. We add functions to classes regularly. We add concepts. We even subtly change the results of things like "begin".
>
> In addition, this proposal needs more bake time, particularly since it has substantial chance of breaking things.
>
> — Weakly Against

> Changed from Weakly Favor [at the telecon] to Strongly against here. I think the best option is for us to deprecate `std::ranges::istream_view` and add `std::views::istream` in its place.
>
> — Strongly Against

§ 3.4. Poll 4: [P2351R0] Mark All Library Static Cast Wrappers As `[[nodiscard]]`

> I encourage explicit `[[nodiscard]]` markings in the standard -- they help catch real world bugs!
>
> — Strongly Favor

> This matches existing practice and make the intent of these function clearer. It has a very low burden on the standard and implementers.
>
> — Strongly Favor

> Although [I] would prefer `[[nodiscard]]` be applied liberally to the standard library, [I] would also be satisfied with it as [Quality of Implementation (QoI)].
>
> — Weakly Favor

> While the direct value of including attributes with no reliable, normative effect in the standard is questionable, there is a very real indirect effect: if implementations can expect that other implementations will issue warnings for the same code, they are less likely to withhold the warnings for compatibility reasons.
>
> — Weakly Favor

> I find the whole idea of `[[nodiscard]]` backwards. The compiler should warn by default on discarded return values. I should have to do something special to get it to stop emitting a warning. If the compiler is doing its job, then nothing should need `[[nodiscard]]` decoration. On the other hand, I wouldn't mind if the standard library did this.
>
> — Neutral

> I am generally in favour of tightening the requirements of [Quality of Implementation (QoI)] to standardize high quality implementations. However In this case I think we can get the result with out prescribing all details.
>
> — Neutral

> I think blanket wording saying which functions should be treated as nodiscard would be superior to explicitly adding it to individual declarations.
>
> — Weakly Against

> It seems a waste of valuable LWG time to forward `[[nodiscard]]` papers without first having reached agreement on whether we want such annotations in the standard at all. I also have serious reservations about the process employed where the mailing list discussion was supposedly tabled until the end of the year, then a month later the paper is suddenly submitted to electronic polling.
>
> — Strongly Against

> - Has no normative effect.
> - Poor use of LEWG time (and it won't stop with just this `[[nodiscard]]` paper).
> - Poor use of WG21 time.
> - There are many (if not all) bucket 3 items I would much rather have before this.
> - No general rules for applying `[[nodiscard]]`.
> - Moves the standard towards being prescriptive, not descriptive.
> - Much better to have a standing document of these things instead of littering the standard with them.
> - Controversial papers should not skip LEWG telecons and go straight to electronic polling.
> - This is something implementors should be doing instead of spending valuable committee time legislating it.
>
> — Strongly Against

3.5. Poll 5: [P2291R2] Add `constexpr` Modifiers To Functions `to_chars` And `from_chars` For Integral Types In `<charconv>` Header

> Having finally specified these most basic operations in a way that is appropriate for the language, it only makes sense that they would be made available for compile time programming. The lack of floating-point support is an obviously necessary concession to practicality which can hopefully be eventually addressed.
>
> — Strongly Favor

> According to implementers this is not hard to provide and would allow constexpr format in the future, which is an useful feature to have. It would certainly help some constexpr libraries like CTRE, etc.
>
> — Strongly Favor

> This is a step in the right direction, we should make functionality available in constexpr context unless there is a good reason not to.
>
> — Strongly Favor

> This seems straightforward, and it has been tested, and it's the kind of thing you would expect to be able to do at compile time.
>
> — Weakly Favor

> I know that this change will be a welcome one for implementers of libraries like CTRE. However, I'm concerned that this will require implementation heroics to prevent large compile time costs for all files that only include `<charconv>` for use at runtime. Although there are obvious approaches that could be used to mitigate this, the paper doesn't discuss the additional compile time overhead of moving integral `to_chars`/`from_chars` into the header nor possible mitigations.
>
> — Neutral

§ References

§ Informative References

[P0592R4]
Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: https://wg21.link/p0592r4

[P2291R2]
Daniil Goncharov, Karaev Alexander. *Add Constexpr Modifiers to Functions to_chars and from_chars for Integral Types in Header*. 17 August 2021. URL: https://wg21.link/p2291r2

[P2351R0]
Hana Dusíková. *Mark all library static cast wrappers as [[nodiscard]]*. 25 April 2021. URL: https://wg21.link/p2351r0

[P2415R1]
Barry Revzin, Tim Song. *What is a view?*. 16 August 2021. URL: https://wg21.link/p2415r1

[P2418R0]
Victor Zverovich. *Add support for std::generator-like types to std::format*. 8 August 2021. URL: https://wg21.link/p2418r0

[P2432R0]
Nicolai Josuttis. *Fixing istream_view*. 27 August 2021. URL: https://wg21.link/p2432r0

[P2436R0]

Bryce Adelstein Lelbach. *2021 September Library Evolution Polls*. 14 September 2021. URL: https://wg21.link/p2436r0